



DEPARTMENT OF COMPUTING

CS-360: Cyber Security

Class: BSCS-2K23

Lab 08: End-to-End Side-Channel Analysis

CLO-3: Students will be able to perform practical technical tasks by following, applying, and demonstrating guided procedures and techniques in supervised laboratory or training environments.

Date: 17th March 2026

Time: 10:00 - 12:50 / 14:00 - 16:50

Lab Instructor: Ms. Hadia Tahir

Class Instructor: Dr. Madiha Khalid



Lab 08: End-to-End Side-Channel Analysis

Introduction

This lab serves as the capstone exercise of the Side-Channel Analysis (SCA) module. It integrates the concepts and techniques learned in Lab 06 (Deep Learning for SCA) and Lab 07 (Trace Generation using Rainbow) to build a complete end-to-end SCA pipeline. In this lab, students will first generate their own simulated power consumption traces from a cryptographic implementation and then apply deep learning techniques to analyze these traces in order to recover a secret cryptographic key. By completing this lab, students will gain hands-on experience with the entire side-channel attack workflow, including trace generation, dataset preparation, model training, and key recovery.

The lab is divided into two main parts.

Part 1 – Trace Generation (Rainbow):

Students will implement a simple cryptographic algorithm and use the Rainbow simulator to generate power consumption traces during encryption. These traces simulate the leakage that occurs in real cryptographic devices.

Part 2 – Deep Learning Attack (TensorFlow):

Using the generated traces, students will train a neural network model to analyze the side-channel leakage and attempt to recover the secret encryption key.

Objectives

After completing this lab, students will be able to:

- Implement a 128-bit XOR cipher in C for ARM architecture
- Generate synthetic power consumption traces using Rainbow framework
- Create custom side-channel analysis datasets in HDF5 format
- Apply Hamming Weight leakage model to XOR operations
- Build and train deep learning models for side-channel attacks
- Perform key recovery on both fixed-key and variable-key scenarios
- Compare attack difficulty between fixed and variable keys
- Understand the complete SCA workflow: code → traces → attack → key

Tools/Software Requirements

- Ubuntu 22.04+ or Kali Linux
- Python 3.10+
- TensorFlow 2.x / Keras
- Rainbow framework (will be installed) — *Note: Rainbow v2.0 introduced breaking API changes. If trace generation fails, use Unicorn directly as a replacement.*
- Unicorn CPU emulator
- Lief (Library to Instrument Executable Formats)
- SCARED (Side-Channel Analysis for Reverse Engineering)



Section 1: Implementing 128-bit XOR Cipher

Understanding the XOR Operation

XOR (exclusive OR) is a fundamental bitwise operation that outputs true (1) only when inputs differ:

XOR Truth Table:

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Key Mathematical Properties:

- Identity: $A \oplus 0 = A$
- Self-Inverse: $A \oplus A = 0$
- Commutative: $A \oplus B = B \oplus A$
- Associative: $(A \oplus B) \oplus C = A \oplus (B \oplus C)$
- Reversibility: $(A \oplus B) \oplus B = A$

Cryptographic Application:

Encryption: $C = P \oplus K$

Decryption: $P = C \oplus K = (P \oplus K) \oplus K = P$

Byte-by-Byte Processing for 128-bit Cipher

The 128-bit XOR cipher processes data in 16-byte blocks, applying XOR independently to each byte:

Processing Algorithm:

For each byte position i from 0 to 15:

$ciphertext[i] = plaintext[i] \oplus key[i]$

Example with First Four Bytes:

Byte 0:

$plaintext[0] = 0x00 = 00000000$ (binary)

$key[0] = 0xDE = 11011110$ (binary)

----- XOR

$ciphertext[0] = 0xDE = 11011110$ (binary)

Byte 1:

$plaintext[1] = 0x11 = 00010001$ (binary)

$key[1] = 0xAD = 10101101$ (binary)

----- XOR

$ciphertext[1] = 0xBC = 10111100$ (binary)

Byte 2:

$plaintext[2] = 0x22 = 00100010$ (binary)



$key[2] = 0xBE = 10111110$ (binary)
----- XOR
 $ciphertext[2] = 0x9C = 10011100$ (binary)

Side-Channel Leakage in XOR Operations

Each XOR operation produces a result with a specific Hamming Weight (number of '1' bits) that correlates with power consumption:

Why XOR Operations Leak Information:

Hamming Weight Definition:

$HW(n)$ = count of '1' bits in binary representation

Example 1 - High Power Consumption:

$plaintext[0] = 0x00 = 00000000$
 $key[0] = 0xDE = 11011110$
 $result = 0xDE = 11011110$
 $HW(0xDE) = 6$ bits set to '1'
→ Higher power consumption (6 bits active)

Example 2 - Low Power Consumption:

$plaintext[0] = 0xFF = 11111111$
 $key[0] = 0xDE = 11011110$
 $result = 0x21 = 00100001$
 $HW(0x21) = 2$ bits set to '1'
→ Lower power consumption (only 2 bits active)

Leakage Principle:

$Power \propto$ Hamming Weight of intermediate values

By observing power differences, attackers can infer the secret key!

Implementation Guidance:

Required Functions:

1. xor_encrypt_128()

Purpose: Perform byte-by-byte XOR on 16-byte blocks
Inputs: `uint8_t plaintext[16], uint8_t key[16]`
Output: `uint8_t ciphertext[16]`
Operation: Loop through bytes, apply XOR

2. hamming_weight()

Purpose: Count number of '1' bits in a byte
Input: `uint8_t value`



Output: `int` (0 to 8)
Method: Count set bits using bitwise operations

3. `main()` or entry function

Purpose: Initialize data and call encryption
Setup: Plaintext and key arrays
Call: `xor_encrypt_128()`

Students must write the actual C code implementation themselves!

Compilation for ARM Architecture

The C code must be compiled for ARM Cortex-M3, as practiced in Lab 07:

Compilation Requirements:

Compiler: `gcc-arm-none-eabi`
Target arch: `ARMv7-M` (`-march=armv7-m`) [`not -mcpu=cortex-m3`]
Instruction: Thumb mode (`-mthumb`)
Optimization: None (`-O0`) for clear trace visibility
Output: ELF executable format
CRITICAL FLAG: `-Wl,-Ttext=0x08000000` (places code at STM32 flash base)

Required Elements:

- Proper architecture and CPU flags
- Use `-Wl,-Ttext=0x08000000` to set flash base (replaces a full linker script)
- Correct function symbols for emulator to find
- No standard library dependencies (`-nostdlib`)

Verification Steps:

- Use 'file' command to verify ELF format
 - Use 'arm-none-eabi-objdump -d' to inspect assembly
 - Confirm XOR and Hamming Weight functions are present
-

Section 2: Generating Power Traces with Rainbow

Note on Rainbow Framework Compatibility: Rainbow v2.0 introduced breaking API changes that affect trace generation. Key issues include: (1) `TraceConfig` parameter renamed from `memory=` to `mem_value=`; (2) `print_asm=` replaced by `print_config=`; (3) `run()` renamed to `start()`; (4) `setup()` must be called explicitly after `load()`. If Rainbow-based trace generation fails, use the Unicorn emulator directly as a functionally equivalent replacement — Unicorn is the underlying engine that Rainbow uses.

Dataset Specifications

Required Datasets:

Dataset 1: FIXED-KEY SCENARIO

Profiling Set (for training):

- Number of traces: 50,000



- Key: Same fixed key for all traces
- Plaintext: Different random plaintext per trace
- Purpose: Train neural network on this key

Attack Set (for evaluation):

- Number of traces: 10,000
- Key: Same fixed key as profiling set
- Plaintext: Different random plaintext per trace
- Purpose: Test if we can recover the key

Dataset 2: VARIABLE-KEY SCENARIO

Profiling Set (for training):

- Number of traces: 50,000
- Key: Different random key per trace (all unique)
- Plaintext: Different random plaintext per trace
- Purpose: Train model to generalize across keys

Attack Set (for evaluation):

- Number of traces: 10,000
- Key: Different random key per trace (never seen before)
- Plaintext: Different random plaintext per trace
- Purpose: Test generalization ability

Hamming Weight Leakage Model Application

Power Trace Generation Using Hamming Weight Model:

For each encryption operation:

Step 1: Execute XOR for each byte

For $i = 0$ to 15:

$$\text{intermediate}[i] = \text{plaintext}[i] \oplus \text{key}[i]$$

Step 2: Calculate Hamming Weight

For $i = 0$ to 15:

$$\text{hw}[i] = \text{count_ones}(\text{intermediate}[i])$$

$$\text{where } \text{hw}[i] \in \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$$

Step 3: Simulate Power Consumption

For $i = 0$ to 15:

$$\text{power_sample}[i] = \text{hw}[i] + \text{noise (optional)}$$

Result: Power trace with 16 time points (one per byte position)

Optional Noise Addition:

- Gaussian noise: $N(0, \sigma^2)$
- Start without noise for initial testing
- Add noise later to test robustness
- Typical σ values: 0.5 to 2.0



HDF5 Dataset Structure

All traces must be stored in HDF5 format following the ASCAD standard:

Required HDF5 File Organization:

File Structure:

```
/Profiling_traces/  
  /traces      Dataset: [50000, 1551] float32  
  /metadata/  
    /plaintext Dataset: [50000, 16] uint8  
    /key        Dataset: [50000, 16] uint8  
    /ciphertext Dataset: [50000, 16] uint8  
  
/Attack_traces/  
  /traces      Dataset: [10000, 1551] float32  
  /metadata/  
    /plaintext Dataset: [10000, 16] uint8  
    /key        Dataset: [10000, 16] uint8  
    /ciphertext Dataset: [10000, 16] uint8
```

Data Specifications:

- traces: Power values (HW-based), float32 type
- plaintext/key/ciphertext: Byte arrays, uint8 type
- Shape [N, 1551]: N traces, 1551 samples per trace (Unicorn emulation produces 1551 HW samples for this cipher)

Expected File Size:

- 60,000 traces $\times$ 1551 samples $\times$ 4 bytes (float32) $\approx$ 372 MB (traces)
 - 60,000 traces $\times$ 16 bytes $\times$ 3 arrays $\approx$ 2.9 MB (metadata)
 - Total per dataset: $\sim$ 357 MB
-

Section 3: Deep Learning Attack on Generated Traces

Label Generation for XOR Cipher

Labels are generated from the intermediate XOR values:

Label Generation Process:

Target Intermediate Value (for each trace):

$$\text{intermediate} = \text{plaintext}[\text{target_byte}] \oplus \text{key}[\text{target_byte}]$$

Two Leakage Models:

1. Hamming Weight (HW) Model - 9 Classes:

$$\text{label} = \text{HW}(\text{plaintext}[\text{target_byte}] \oplus \text{key}[\text{target_byte}])$$

where $\text{label} \in \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$

2. Identity (ID) Model - 256 Classes:



$label = plaintext[target_byte] \oplus key[target_byte]$
where $label \in \{0, 1, 2, \dots, 255\}$

Example (attacking byte 0 with HW model):

Trace #0:

$plaintext[0] = 0x00$
 $key[0] = 0xDE$
 $intermediate = 0x00 \oplus 0xDE = 0xDE = 11011110$
 $label = HW(0xDE) = 6$

Trace #1:

$plaintext[0] = 0xFF$
 $key[0] = 0xDE$
 $intermediate = 0xFF \oplus 0xDE = 0x21 = 00100001$
 $label = HW(0x21) = 2$

Deep Learning Attack Workflow

The attack proceeds through several stages:

Stage 1: Dataset Loading and Preparation

- Load HDF5 file using h5py
- Extract profiling traces and metadata
- Extract attack traces and metadata
- Verify data shapes match expectations
- Check data types (float32 for traces, uint8 for metadata)

Stage 2: Label Generation for Training

- For each profiling trace, compute intermediate value
- Apply chosen leakage model (HW or ID)
- Convert labels to categorical format (one-hot encoding)
- Do NOT generate labels for attack set (pretend unknown)

Stage 3: Model Architecture Design

Model Architecture Guidance:

Fixed-Key Scenario:

Input Layer: 1551 neurons (1551 power samples per trace)

Hidden Layer 1: 128-256 neurons, ReLU activation

Hidden Layer 2: 128-256 neurons, ReLU activation

Output Layer: 9 neurons (HW), Softmax activation

Training: 50-75 epochs

Batch Size: 200-500

Optimizer: Adam



Loss: Categorical Crossentropy
Expected Acc: 40-60% on validation set

Variable-Key Scenario (needs deeper network):

Input Layer: 1551 neurons (matches trace length)
Hidden Layer 1: 256-512 neurons, ReLU
Hidden Layer 2: 256-512 neurons, ReLU
Hidden Layer 3: 128-256 neurons, ReLU (additional depth)
Output Layer: 9 neurons, Softmax

Training: 75-100 epochs
Batch Size: 200-500
Optimizer: Adam
Loss: Categorical Crossentropy
Optional: Dropout (0.2-0.3) for regularization
Expected Acc: 30-50% on validation set

Stage 4: Training Process

- Split profiling data: 80% training, 20% validation
- Use stratified split to maintain class distribution
- Train model with specified epochs and batch size
- Monitor both training and validation accuracy
- Watch for overfitting (validation accuracy plateau/decrease)
- Save best model based on validation accuracy
- Plot training history for analysis

Key Recovery Attack Algorithm

Attack Algorithm:

Goal: Recover target key byte from attack traces

For each possible key byte guess $k = 0$ to 255:

Initialize: $score[k] = 0$

For each attack trace $i = 0$ to N_{attack} :

1. Get plaintext byte:
 $p = \text{plaintext}[i][\text{target_byte}]$
2. Compute hypothetical intermediate value:
 $\text{intermediate_hypothesis} = p \oplus k$
3. Compute hypothetical Hamming Weight:
 $\text{hw_hypothesis} = \text{HW}(\text{intermediate_hypothesis})$
4. Get model's probability prediction:
 $\text{prob} = \text{model_prediction}[i][\text{hw_hypothesis}]$



5. Accumulate log probability:
 $score[k] += \log(prob + \epsilon)$
where $\epsilon = 1e-36$ (to avoid $\log(0)$)

End For

End For

Key Recovery:

$recovered_key = \text{argmax}(score)$
(key guess with highest accumulated score)

Evaluation:

1. Sort all 256 keys by score (descending)
 2. Find position of correct key in sorted list
 3. $key_rank = 0$ means perfect recovery
 4. $success_rate = \text{percentage of bytes with rank} = 0$
-

Success Metrics and Expected Results

Evaluation Metrics:

1. Key Rank:

Position of correct key in sorted candidates
Best: $rank = 0$ (correct key is top guess)
Worst: $rank = 255$ (correct key is last guess)

2. Success Rate:

Percentage of key bytes recovered with $rank = 0$
Fixed-key target: $>90\%$
Variable-key target: $>20\%$ ($>50\%$ excellent)

3. Guessing Entropy:

Median key rank across multiple experiments
Lower is better

4. Number of Traces Required:

Minimum traces needed to achieve $rank = 0$
Measure attack efficiency



Lab Tasks

Phase 1: Implementation and Trace Generation

Task 1: XOR Cipher Implementation in C

Develop and compile a 128-bit XOR cipher implementation for the ARM architecture.

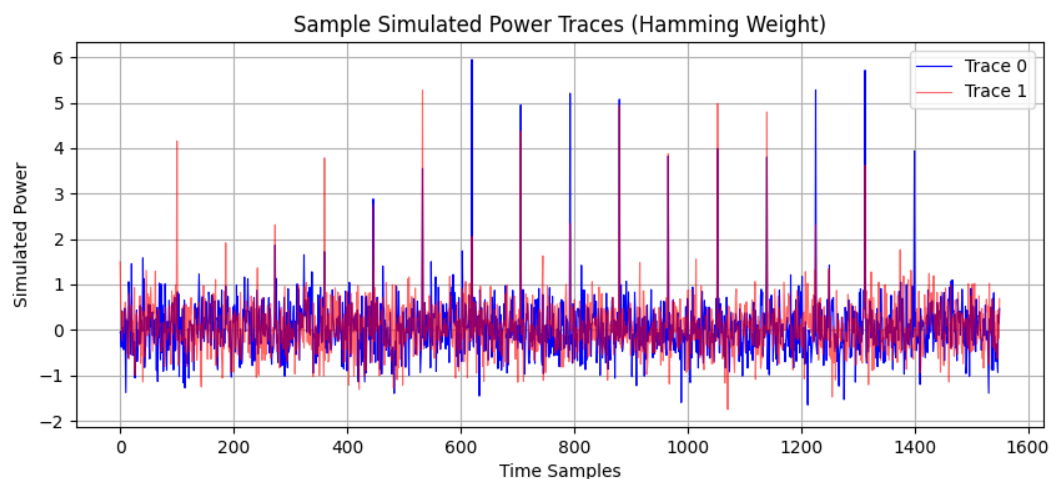
- Write a C program implementing a 128-bit XOR encryption function that processes plaintext and key arrays byte-by-byte.
- Implement a Hamming Weight function to count the number of '1' bits in a byte.
- Properly initialize plaintext, key, and ciphertext arrays.
- Compile the program for ARM Cortex-M3 architecture.

```
(sca_env)kali@kali: ~  
Session Actions Edit View Help  
└─$  
(sca_env)-(kali@kali)-[~]  
└─$ nano xor_cipher.c  
(sca_env)-(kali@kali)-[~]  
└─$ arm-none-eabi-gcc -march=armv7-m -mthumb -O0 -g -nostdlib -Wl,-Ttext=0x08000000 xor_cipher.c -o xor_cipher.elf  
(sca_env)-(kali@kali)-[~]  
└─$ file xor_cipher.elf  
xor_cipher.elf: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked, with debug_info, not stripped
```

Task 2: Fixed-Key Dataset Generation

Generate a side-channel dataset where the same secret key is used for all traces.

- Develop a Python script using the Rainbow framework to simulate encryption execution.
- Generate 50,000 profiling traces using random plaintexts and a single fixed key.
- Generate 10,000 attack traces using the same key but different random plaintexts.
- Apply the Hamming Weight leakage model to simulate power consumption during XOR operations.
- Store traces and metadata (plaintext, key, ciphertext) in HDF5 format following an ASCAD-like structure.
- Verify dataset dimensions: profiling ($50,000 \times 16$) and attack ($10,000 \times 16$).
- Visualize a few sample traces to confirm that the dataset was generated correctly.
- Record the fixed key value used during dataset generation for later verification.



Task 3: Variable-Key Dataset Generation

Extend the trace generation process to support random keys for each trace.

- Modify the dataset generation script so that a new random key is generated for every encryption.
- Generate 50,000 profiling traces with random plaintexts and random keys.
- Generate 10,000 attack traces with different random keys.
- Ensure that keys are not reused across the entire dataset.
- Apply the same Hamming Weight leakage model used in the fixed-key scenario.
- Store the dataset in the same HDF5 structure as the fixed-key dataset.

```
(sca_env)-(kali@kali)-[~]  
$ nano generate_variable_key_sim.py  
  
(sca_env)-(kali@kali)-[~]  
$ python3 generate_variable_key_sim.py  
Generating Variable-Key Dataset via Simulation ...  
  → Generating 50000 profiling traces ...  
  → Generating 10000 attack traces ...  
Saving to variable_key_dataset.h5 in ASCAD format ...  
Variable-key dataset generated successfully!  
  
(sca_env)-(kali@kali)-[~]  
$
```



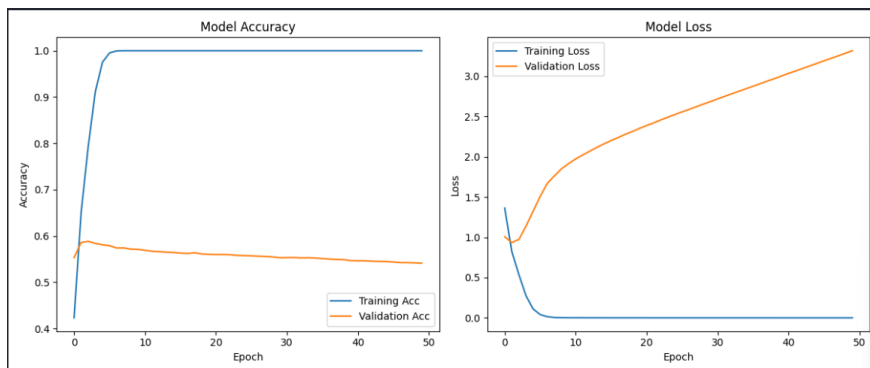
Phase 2: Deep Learning Attacks

Task 4: Fixed-Key Attack Implementation

Perform a deep learning-based side-channel attack using the fixed-key dataset.

- Load the generated HDF5 dataset containing profiling and attack traces.
- Generate Hamming Weight labels for the selected target byte (begin with byte 0).
- Build a Multilayer Perceptron (MLP) model suitable for side-channel classification.
- Split profiling data into 80% training and 20% validation sets.
- Train the model for 50–75 epochs using an appropriate batch size.
- Monitor training performance and plot loss and accuracy curves.
- Save the trained model for later use.
- Implement the key recovery procedure and apply it to the attack traces.
- Compute the recovered key byte, key rank, and attack success rate.

```
(sca_env)kali@kali: ~  
Session Actions Edit View Help  
82/157 0s 7ms/step - accuracy: 1.0000 - loss: 6.3932e-0  
90/157 0s 7ms/step - accuracy: 1.0000 - loss: 6.3922e-0  
99/157 0s 7ms/step - accuracy: 1.0000 - loss: 6.3908e-0  
107/157 0s 7ms/step - accuracy: 1.0000 - loss: 6.3890e-0  
115/157 0s 7ms/step - accuracy: 1.0000 - loss: 6.3870e-0  
123/157 0s 7ms/step - accuracy: 1.0000 - loss: 6.3845e-0  
131/157 0s 7ms/step - accuracy: 1.0000 - loss: 6.3820e-0  
139/157 0s 7ms/step - accuracy: 1.0000 - loss: 6.3792e-0  
146/157 0s 7ms/step - accuracy: 1.0000 - loss: 6.3770e-0  
155/157 0s 7ms/step - accuracy: 1.0000 - loss: 6.3744e-0  
157/157 1s 8ms/step - accuracy: 1.0000 - loss: 6.3248e-0  
6 - val_accuracy: 0.5412 - val_loss: 3.3183  
Model saved to 'fixed_key_model.keras'.  
Training curves saved to 'fixed_key_training_history.png'.  
  
— Phase 3: Key Recovery Attack —  
313/313 1s 2ms/step  
  
===== Attack Results =====  
Target Byte Index: 0  
Actual Key Byte: 0xDE  
Recovered Key Byte: 0xDE  
Key Rank: 0 (0 is perfect)  
>> SUCCESS! The model perfectly recovered the secret key byte! <<  
  
(sca_env)-(kali@kali)-[~]  
$
```





Task 5: Variable-Key Attack Implementation

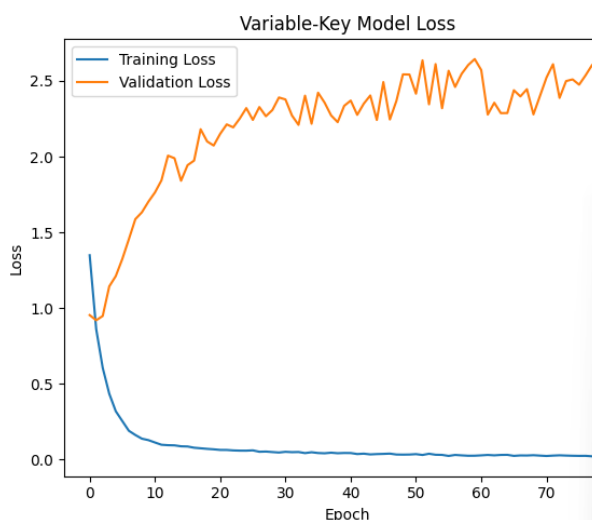
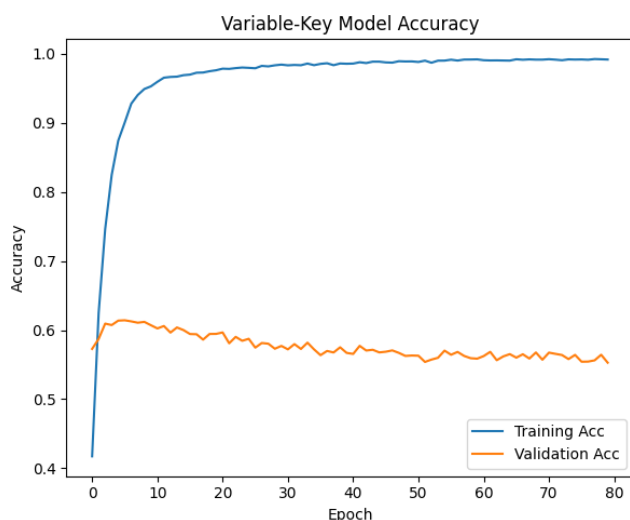
Evaluate model performance when each trace uses a different key.

- Load the variable-key dataset generated in Phase 1.
- Generate labels based on the XOR intermediate value for each trace.
- Build a larger or deeper MLP model to improve generalization across keys.
- Train the model for 75–100 epochs, carefully monitoring validation performance.
- Save the best-performing model during training.
- Perform attacks on each attack trace individually, since each trace uses a different key.
- Calculate the overall attack success rate and key rank distribution.
- Analyze traces where the attack fails and identify possible reasons.
- Compare the model behavior with results from the fixed-key scenario.

```
— Phase 3: Individual Trace Key Recovery Attack —
313/313 1s 2ms/step
Evaluating attack success rate across 10,000 unique keys ...
100%|████████████████████| 10000/10000 [00:00<00:00, 1249420.32it/s]

===== Attack Results =====
Total Traces Evaluated: 10000
Successful Recoveries: 174 (Rank = 0)
Success Rate: 1.74%
Guessing Entropy: 38.0 (Median Key Rank)

(sca_env)-(kali@kali)-[~]
$
```





Task 6: Comparative Analysis and Reporting

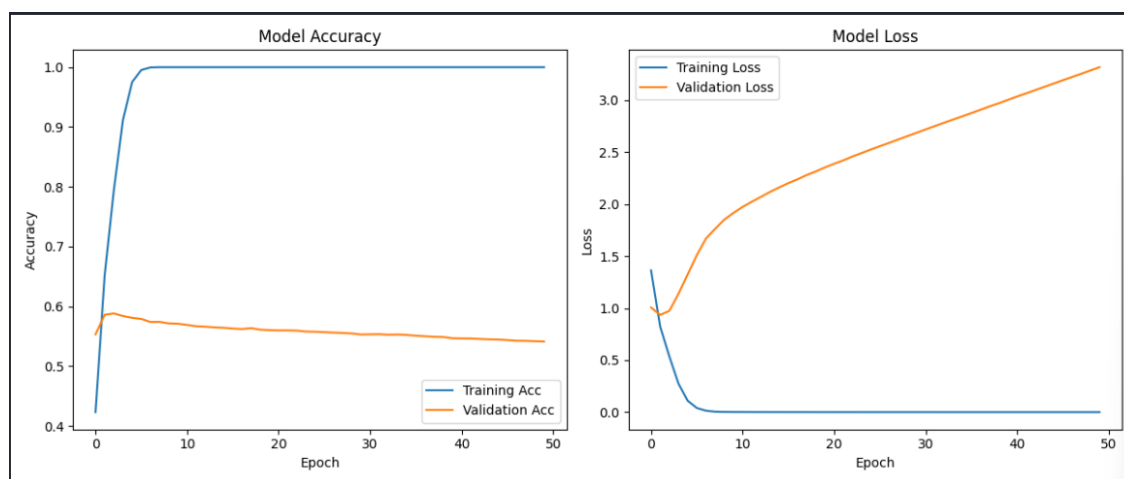
Analyze and compare the results obtained from both attack scenarios.

- Create a comparison table summarizing key metrics for fixed-key and variable-key experiments.

Metric	Fixed-Key Attack	Variable-Key Attack
Attack Goal	Recover a single constant key	Recover 10,000 unique keys
Neural Network Depth	2 Hidden Layers (256 units each)	3 Hidden Layers (512, 512, 256 units)
Validation Accuracy	Plateaued at ~55%	Plateaued at ~55-60%
Guessing Entropy (Median Rank)	0	38.0
Overall Success Rate	100% (Rank 0 achieved)	1.74% (174 keys recovered)

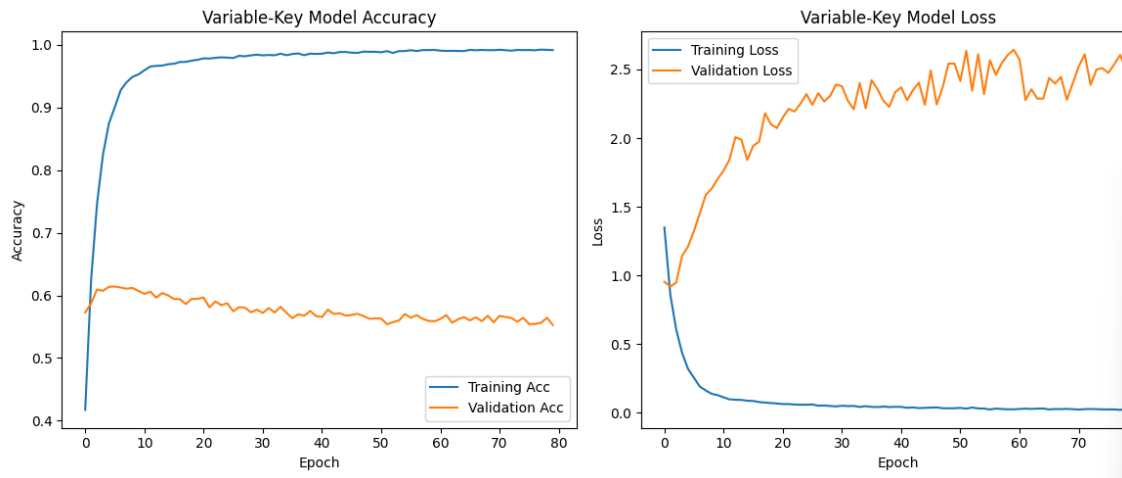
- Plot and compare training curves for both models & Compare attack success rates and key rank distributions.

Fixed-Key Model: The training curves show that the model memorized the training data very quickly, reaching 100% training accuracy within the first 10 epochs. The validation accuracy plateaued around 55%, indicating that the network learned a strong, specific template for the fixed key's power signature.





Variable-Key Model: Despite having a deeper architecture with Dropout regularization to prevent overfitting, the variable-key model struggled to achieve the same predictive power. While training accuracy eventually approached 1.0, the validation loss was significantly higher and more erratic, demonstrating the difficulty of generalizing across 256 different key classes.



- Explain why variable-key attacks are more challenging than fixed-key attacks.

Template Matching vs. Generalization: In a fixed-key scenario, the neural network only needs to learn the specific power footprint of one target key combined with varying plaintexts. It acts as a highly tuned template matcher. In a variable-key scenario, the network must learn the generalized mathematical relationship of the XOR operation and the Hamming weight leakage model across all 256 possible key values simultaneously.

Signal-to-Noise Ratio: Because the key is constantly changing in the variable dataset, the target key is no longer a constant bias in the power consumption data. The model must isolate the exact correlation between the varying plaintext and the varying key amidst simulated environmental noise, making feature extraction significantly more complex for the hidden layers.